

Assignment_03

December 4, 2023

1 Authors

- Raphael Wäspi (18-918-938)
- Nicolas Huber (16-936-205)

```
[159]: import math
# Imports
import numpy as np
import matplotlib.pyplot as plt
import networkx as nx
import os
import glob
import re
import random
from statistics import mean

import scipy.special
from tqdm import tqdm
import shutil
from joblib import Parallel, delayed
from scipy.special import factorial
import powerlaw
from NEMtropy import UndirectedGraph
```

```
[160]: # Define colors with reduced opacity from a colormap
COLOR_BLUE = 'dodgerblue'
COLOR_RED = 'lightcoral'
COLOR_GREY = 'slategrey'
COLOR_BLACK = 'black'
COLOR_LIGHT_GRAY = 'lightgray'
COLOR_DARK_GRAY = 'darkgray'

# Define the main data directory
OUTPUT_PATH = "./output"
```

```
[161]: # Prepare output folder

# Delete the entire 'output' directory and its contents
```

```

if os.path.exists(OUTPUT_PATH):
    shutil.rmtree(OUTPUT_PATH)

# Create the 'output' directory
os.makedirs(OUTPUT_PATH)

# Define subdirectories
OUTPUT_PATH_ER = os.path.join(OUTPUT_PATH, "erdos_renyi_avg_clustering")

# Recreate the subdirectories
os.makedirs(OUTPUT_PATH_ER)

```

```

[162]: # Define the directory containing the .gml files
directory = 'assignment_03_data'

# Use glob to get a list of all .gml files in the directory
gml_files = glob.glob(os.path.join(directory, '*.gml'))

# Init an empty dictionary for all graphs - the key is the actual name, and the
↳ value is the graph data
graphs = {}

# Use tqdm to show a progress bar in Jupyter Notebook
with tqdm(total=len(gml_files), desc="Loading GML Files") as pbar:
    # Loop through the list of .gml files and load them
    for file_path in gml_files:
        with open(file_path, 'r') as file:
            content = file.read()

            # Use regex to extract the desired part
            match = re.search(r'/(~/)+\.gml$', file_path)
            if match:
                graph_name = match.group(1)

                # Add to the graph dictionary
                graph = nx.read_gml(file_path)
                graphs[graph_name] = graph

                # Print the graph name when each part is finished
                print(f"Finished loading {graph_name}")

        pbar.update(1) # Update the progress bar

```

Loading GML Files: 100% | 2/2 [00:00<00:00, 79.99it/s]

```
[163]: # Define the directory containing the .gml files
directory = 'assignment_03_data/World_Trade_Web'

# Use glob to get a list of all .gml files in the directory
gml_files = glob.glob(os.path.join(directory, '*.graphml'))

# Init an empty dictionary for all graphs - the key is the actual name, and the
↳ value is the graph data
graphs_wtn = {}

# Use tqdm to show a progress bar in Jupyter Notebook
with tqdm(total=len(gml_files), desc="Loading GML WTW Files") as pbar:
    # Loop through the list of .gml files and load them
    for file_path in gml_files:
        with open(file_path, 'r') as file:
            content = file.read()

            # Use regex to extract the desired part
            match = re.search(r'/(~/+)\.txt.graphml$', file_path)
            if match:
                graph_name = match.group(1)

                # Add to the graph dictionary
                graph = nx.read_graphml(file_path)
                graphs_wtn[graph_name] = graph

                # Print the graph name when each part is finished
                print(f"Finished loading {graph_name}")

        pbar.update(1) # Update the progress bar
```

```
Loading GML WTW Files:  9%|          | 1/11 [00:00<00:01,  7.14it/s]
Finished loading World_Trade_Web\WDN_1992
Loading GML WTW Files: 18%|          | 2/11 [00:00<00:02,  4.01it/s]
Finished loading World_Trade_Web\WDN_1993
Finished loading World_Trade_Web\WDN_1994
Loading GML WTW Files: 36%|          | 4/11 [00:00<00:01,  3.73it/s]
Finished loading World_Trade_Web\WDN_1995
Loading GML WTW Files: 45%|          | 5/11 [00:01<00:01,  3.36it/s]
Finished loading World_Trade_Web\WDN_1996
Loading GML WTW Files: 55%|          | 6/11 [00:01<00:01,  3.10it/s]
Finished loading World_Trade_Web\WDN_1997
Loading GML WTW Files: 64%|          | 7/11 [00:02<00:01,  2.97it/s]
```

```

Finished loading World_Trade_Web\WDN_1998
Loading GML WTW Files: 73%|          | 8/11 [00:02<00:01, 2.86it/s]
Finished loading World_Trade_Web\WDN_1999
Loading GML WTW Files: 82%|          | 9/11 [00:02<00:00, 2.77it/s]
Finished loading World_Trade_Web\WDN_2000
Loading GML WTW Files: 91%|          | 10/11 [00:03<00:00, 2.54it/s]
Finished loading World_Trade_Web\WDN_2001
Loading GML WTW Files: 100%|         | 11/11 [00:03<00:00, 2.96it/s]
Finished loading World_Trade_Web\WDN_2002

```

```

[164]: # Define the directory containing the .npz files
directory = 'assignment_03_data/NYSE'

# Use glob to get a list of all .npz files in the directory
npz_files = glob.glob(os.path.join(directory, '*.npz'))

# Init an empty dictionary for all arrays - the key is the actual name, and the
# value is the array data
arrays_nyse = {}

# Use tqdm to show a progress bar in Jupyter Notebook
with tqdm(total=len(npz_files), desc="Loading NPZ Files") as pbar:
    # Loop through the list of .npz files and load them
    for file_path in npz_files:
        array_name = os.path.splitext(os.path.basename(file_path))[0]

        # Load the numpy array
        array_data = np.load(file_path)

        # Add to the array dictionary
        arrays_nyse[array_name] = array_data

        # Print the array name when each part is finished
        print(f"Finished loading {array_name}")

        pbar.update(1) # Update the progress bar

# Get stocknames
stocks = np.loadtxt(directory+"/stocknames.txt", dtype=str)

```

```

Loading NPZ Files: 100%|          | 6/6 [00:00<00:00, 103.44it/s]

```

```

Finished loading cormat_1h
Finished loading cormat_1m
Finished loading cormat_gaussian_1h
Finished loading cormat_gaussian_1m
Finished loading cormat_onefactor_1h
Finished loading cormat_onefactor_1m

```

2 Stock correlations

2.1 1. Is there a market mode at the minute and hour timescale?

Hint: The existence of the market mode is related to the gap between the largest and second largest eigenvalues of the correlation matrix.

```

[165]: # Load Data
arrays_nyse

```

```

[165]: {'cormat_1h': array([[ 1.          ,  0.16724079,  0.17792163, ...,  0.0185428 ,
    -0.04161977,  0.157766  ],
    [ 0.16724079,  1.          ,  0.46343185, ...,  0.28985369,
    0.20418909,  0.29171091],
    [ 0.17792163,  0.46343185,  1.          , ...,  0.39057488,
    0.24218336,  0.24357246],
    ...,
    [ 0.0185428 ,  0.28985369,  0.39057488, ...,  1.          ,
    0.42720044,  0.44063307],
    [-0.04161977,  0.20418909,  0.24218336, ...,  0.42720044,
    1.          ,  0.41781517],
    [ 0.157766  ,  0.29171091,  0.24357246, ...,  0.44063307,
    0.41781517,  1.          ]]),
    'cormat_1m': array([[1.          ,  0.16174328,  0.29933708, ...,  0.21205377,
    0.26330366,
    0.28766365],
    [0.16174328,  1.          ,  0.2303112 , ...,  0.18402302,  0.23935938,
    0.35348702],
    [0.29933708,  0.2303112 ,  1.          , ...,  0.2000015 ,  0.27890206,
    0.29681243],
    ...,
    [0.21205377,  0.18402302,  0.2000015 , ...,  1.          ,  0.30616488,
    0.32896516],
    [0.26330366,  0.23935938,  0.27890206, ...,  0.30616488,  1.          ,
    0.35136225],
    [0.28766365,  0.35348702,  0.29681243, ...,  0.32896516,  0.35136225,
    1.          ]]),
    'cormat_gaussian_1h': array([[ 1.          ,  0.08634609, -0.03718647, ...,
    0.08663246,

```

```

-0.00658567, -0.0686738 ],
[ 0.08634609, 1.          , 0.01719712, ..., 0.03085411,
  0.06973086, -0.07109787],
[-0.03718647, 0.01719712, 1.          , ..., 0.10206038,
  -0.03276127, 0.02083412],
...,
[ 0.08663246, 0.03085411, 0.10206038, ..., 1.          ,
  -0.01918115, 0.01378459],
[-0.00658567, 0.06973086, -0.03276127, ..., -0.01918115,
  1.          , -0.08382412],
[-0.0686738 , -0.07109787, 0.02083412, ..., 0.01378459,
  -0.08382412, 1.          ]]),
'cormat_gaussian_1m': array([[ 1.00000000e+00,  3.37684266e-03,
  7.23562750e-03, ...,
    1.92249770e-03, -9.75349240e-03, -7.36813726e-03],
[ 3.37684266e-03,  1.00000000e+00, -1.13536042e-02, ...,
    1.23984767e-03, -7.55194316e-03,  1.26137140e-03],
[ 7.23562750e-03, -1.13536042e-02,  1.00000000e+00, ...,
   -3.74411178e-03, -1.00702836e-02, -3.60383913e-03],
...,
[ 1.92249770e-03,  1.23984767e-03, -3.74411178e-03, ...,
    1.00000000e+00,  9.47256488e-03,  6.82972396e-04],
[-9.75349240e-03, -7.55194316e-03, -1.00702836e-02, ...,
    9.47256488e-03,  1.00000000e+00,  1.60163843e-02],
[-7.36813726e-03,  1.26137140e-03, -3.60383913e-03, ...,
    6.82972396e-04,  1.60163843e-02,  1.00000000e+00]]),
'cormat_onefactor_1h': array([[1.          , 0.04869414, 0.27441816, ...,
  0.16749489, 0.03936576,
    0.16454019],
[0.04869414, 1.          , 0.22243747, ..., 0.18359562, 0.03585145,
    0.30874494],
[0.27441816, 0.22243747, 1.          , ..., 0.18981075, 0.02485402,
    0.37026938],
...,
[0.16749489, 0.18359562, 0.18981075, ..., 1.          , 0.00679392,
    0.21172785],
[0.03936576, 0.03585145, 0.02485402, ..., 0.00679392, 1.          ,
    0.08217892],
[0.16454019, 0.30874494, 0.37026938, ..., 0.21172785, 0.08217892,
    1.          ]]),
'cormat_onefactor_1m': array([[1.          , 0.0778179 , 0.1258295 , ...,
  0.10320433, 0.12333375,
    0.15272495],
[0.0778179 , 1.          , 0.13371086, ..., 0.12449381, 0.11540375,
    0.17376616],
[0.1258295 , 0.13371086, 1.          , ..., 0.17169689, 0.19273065,
    0.26304523],

```

```
...,
[0.10320433, 0.12449381, 0.17169689, ..., 1.          , 0.16421046,
 0.22870027],
[0.12333375, 0.11540375, 0.19273065, ..., 0.16421046, 1.          ,
 0.23390987],
[0.15272495, 0.17376616, 0.26304523, ..., 0.22870027, 0.23390987,
 1.          ]])}
```

```
[166]: ### NOT SURE IF A LOADED THE RIGHT DATA
cormat_1h = arrays_nyse["cormat_1h"]
cormat_1m = arrays_nyse["cormat_1m"]

# Calculate eigenvalues
eigenvalues_1h = np.linalg.eigvals(cormat_1h)
eigenvalues_1m = np.linalg.eigvals(cormat_1m)

# Sort eigenvalues in descending order
sorted_eigenvalues_1h = np.sort(eigenvalues_1h)[::-1]
sorted_eigenvalues_1m = np.sort(eigenvalues_1m)[::-1]
```

```
[167]: def getAllGapValues(sorted_eigenvalues):
        result = []
        for i in range(len(sorted_eigenvalues)-1):
            result.append(sorted_eigenvalues[i]-sorted_eigenvalues[i+1])

        return result
```

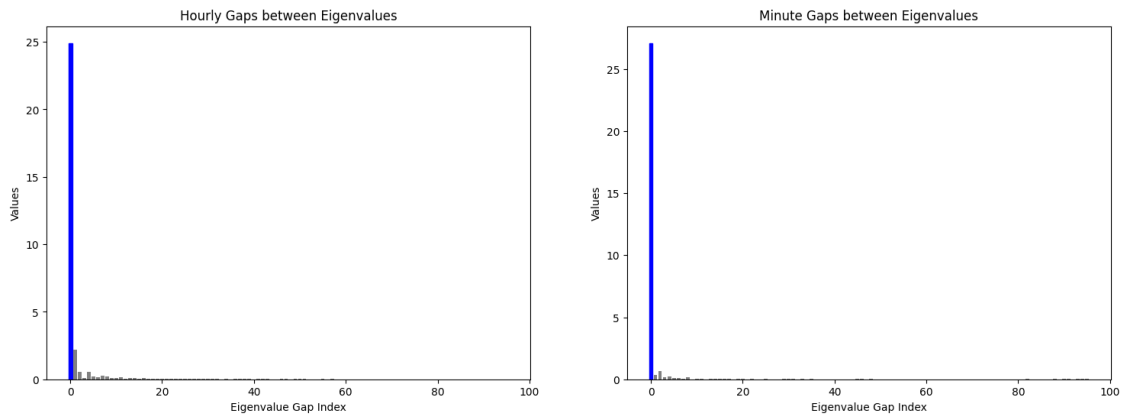
```
[168]: # Get all gaps
hourly_gaps = getAllGapValues(sorted_eigenvalues_1h)
min_gaps = getAllGapValues(sorted_eigenvalues_1m)

# Create a figure with two subplots side by side
fig, axs = plt.subplots(1, 2, figsize=(18, 6))

# Plot the hourly gaps in the first subplot
bars1 = axs[0].bar(range(len(hourly_gaps)), hourly_gaps, color='gray')
bars1[0].set_color('blue')
axs[0].set_xlabel('Eigenvalue Gap Index')
axs[0].set_ylabel('Values')
axs[0].set_title('Hourly Gaps between Eigenvalues')

# Plot the minute gaps in the second subplot
bars2 = axs[1].bar(range(len(min_gaps)), min_gaps, color='gray')
bars2[0].set_color('blue')
axs[1].set_xlabel('Eigenvalue Gap Index')
axs[1].set_ylabel('Values')
axs[1].set_title('Minute Gaps between Eigenvalues')
```

```
plt.show()
```



We can clearly see that the first gap is surpassing the others. This suggests the presence of a market mode at both the hour and minute timescales.

2.2 2. Compute the degree distributions of the 1 hour and 1 minute MSTs. Which are the 5 stocks with highest degree on the two different timescales MST?

Hint: To build the MST, use `networkx.minimum_spanning_tree` on the weighted undirected graph that is generated by the matrix with entries `dij` (see lecture on financial networks).

Assign the correct ticker (AAPL, AMZN, ...) as an attribute to the nodes, by using `networkx.set_node_attributes(G, values=tickers, name='ticker')`

```
[169]: cormat_1h = arrays_nyse["cormat_1h"]
cormat_1m = arrays_nyse["cormat_1m"]
tickers_1h = stocks
tickers_1m = stocks
```

```
[170]: # Use MP to distinguish relevant signal from noise by identifying and removing
      ↪ noise components in the eigenvalues.
def denoise_mp(correlation_matrix, t, noise_constant=1):

    # As said sigma can be assumed to be 1
    sigma = 1

    # Compute eigenvalues
    eigenvalues, eigenvectors = np.linalg.eig(correlation_matrix)

    # Determine the theoretical lambda_max
    #Q = len(correlation_matrix) / len(eigenvalues)
```

```

Q = t / len(correlation_matrix)
lambda_max = (sigma ** 2) * (1 + (1 / Q) + 2 * np.sqrt(1 / Q))

# Any < is classified as noise, others are signal - Replace noise s
↳with a constant
eigenvalues_denoised = np.where(eigenvalues < lambda_max, noise_constant,
↳eigenvalues)

# Renormalize eigenvalues so that sum of all eigenvalues are the length of
↳the array
eigenvalues_denoised = eigenvalues_denoised / sum(eigenvalues_denoised) *
↳len(eigenvalues_denoised)

# Undo diagonalization to get denoised correlation matrix
denoised_matrix = np.dot(eigenvectors, np.dot(np.
↳diag(eigenvalues_denoised), np.linalg.inv(eigenvectors)))

return denoised_matrix

```

```

[171]: cormat_1h_denoised = denoise_mp(cormat_1h, 22*6*60)
cormat_1m_denoised = denoise_mp(cormat_1m, 22*6*60)

num_assets_1h = len(cormat_1h_denoised)
num_assets_1m = len(cormat_1m_denoised)

# Initialize an empty distance matrix
distance_matrix_1h = np.zeros((num_assets_1h, num_assets_1h))
distance_matrix_1m = np.zeros((num_assets_1m, num_assets_1m))

# Compute the distance matrix based on the correlation coefficients
for i in range(num_assets_1h):
    for j in range(i + 1, num_assets_1h):
        correlation_coefficient = cormat_1h_denoised[i, j]
        distance_ij = np.sqrt(2 * (1 - correlation_coefficient))
        distance_matrix_1h[i, j] = distance_ij
        distance_matrix_1h[j, i] = distance_ij # Since these values should be
↳the same (symmetric). However we could only look on the left or right side
↳from the diagonal

# Compute the distance matrix based on the correlation coefficients
for i in range(num_assets_1m):
    for j in range(i + 1, num_assets_1m):
        correlation_coefficient = cormat_1m_denoised[i, j]
        distance_ij = np.sqrt(2 * (1 - correlation_coefficient))
        distance_matrix_1m[i, j] = distance_ij

```

```
distance_matrix_1m[j, i] = distance_ij # Since these values should be
↳ the same (symmetric). However we could only look on the left or right side
↳ from the diagonal
```

[172]: # Prepare the graph data

```
# Create two weighted graphs
G_1h = nx.Graph()
G_1m = nx.Graph()

# Create a dictionary with tickers as both keys and values
tickers_dict_1h = {ticker: ticker for ticker in tickers_1h}
tickers_dict_1m = {ticker: ticker for ticker in tickers_1m}

# Add nodes to the graph
G_1h.add_nodes_from(tickers_1h)
G_1m.add_nodes_from(tickers_1m)

# Set 'ticker' attribute for each node
nx.set_node_attributes(G_1h, tickers_dict_1h, name='ticker')
nx.set_node_attributes(G_1m, tickers_dict_1m, name='ticker')

# Add weighted edges for G_1h
for i in range(len(tickers_1h)):
    for j in range(i + 1, len(tickers_1h)):
        weight_1h = distance_matrix_1h[i][j]
        G_1h.add_edge(tickers_1h[i], tickers_1h[j], weight=weight_1h)

# Add weighted edges for G_1m
for i in range(len(tickers_1m)):
    for j in range(i + 1, len(tickers_1m)):
        weight_1m = distance_matrix_1m[i][j]
        G_1m.add_edge(tickers_1m[i], tickers_1m[j], weight=weight_1m)
```

[173]: # Show it graphically

```
# Make MSTs (Cheapest Path to each node)
mst_1h = nx.minimum_spanning_tree(G_1h)

# Subplot for 1h
plt.figure(figsize=(15, 6))

# Plot the G_1h
plt.subplot(1, 2, 1)
pos = nx.spring_layout(G_1h) # You can choose a different layout if needed
```

```

nx.draw(G_1h, pos, with_labels=True, node_size=800, node_color='skyblue',
        ↪font_weight='bold', edge_color='gray')
plt.title('Graph for 1 hour')

# Plot the mst_1h
plt.subplot(1, 2, 2)
pos = nx.spring_layout(mst_1h) # You can use different layout algorithms
nx.draw(mst_1h, pos, with_labels=True, font_size=8, font_color='black',
        ↪node_size=500, node_color='skyblue', edge_color='gray')
plt.title('1 hour MST')

plt.tight_layout()
plt.show()

# Make MSTs (Cheapest Path to each node)
mst_1m = nx.minimum_spanning_tree(G_1m)

# Subplot for 1h
plt.figure(figsize=(15, 6))

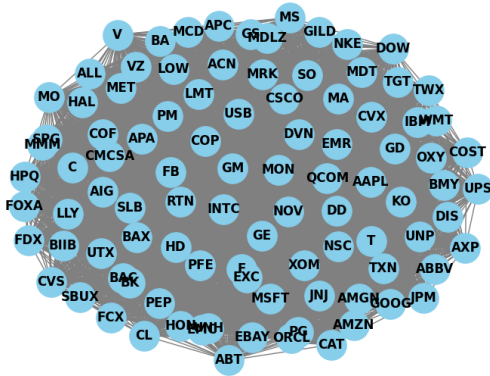
# Plot the G_1m
plt.subplot(1, 2, 1)
pos = nx.spring_layout(G_1m) # You can choose a different layout if needed
nx.draw(G_1m, pos, with_labels=True, node_size=800, node_color='skyblue',
        ↪font_weight='bold', edge_color='gray')
plt.title('Graph for 1 minute')

# Plot the mst_1h
plt.subplot(1, 2, 2)
pos = nx.spring_layout(mst_1m) # You can use different layout algorithms
nx.draw(mst_1m, pos, with_labels=True, font_size=8, font_color='black',
        ↪node_size=500, node_color='skyblue', edge_color='gray')
plt.title('1 minute MST')

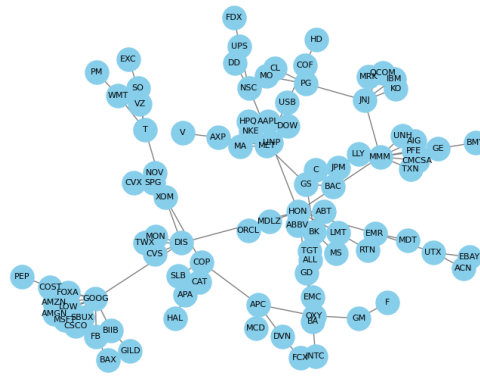
plt.tight_layout()
plt.show()

```

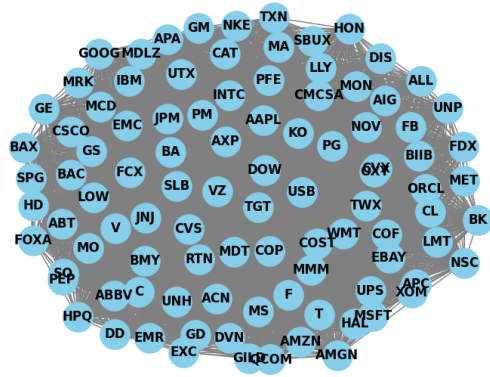
Graph for 1 hour



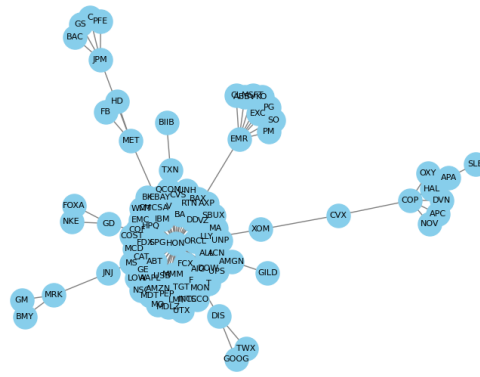
1 hour MST



Graph for 1 minute



1 minute MST



```
[174]: # Print Result

# Compute degree distributions
degree_distribution_1h = dict(mst_1h.degree())
degree_distribution_1m = dict(mst_1m.degree())

# Sort the degree distribution (dict) after descending values
sorted_dict_desc_1h = dict(sorted(degree_distribution_1h.items(), key=lambda
    ↪ item: item[1], reverse=True))
sorted_dict_desc_1m = dict(sorted(degree_distribution_1m.items(), key=lambda
    ↪ item: item[1], reverse=True))

# Print Result
print("Top 5 stocks with highest degree in 1 hour MST:",
    ↪ list(sorted_dict_desc_1h.keys())[:5])
```

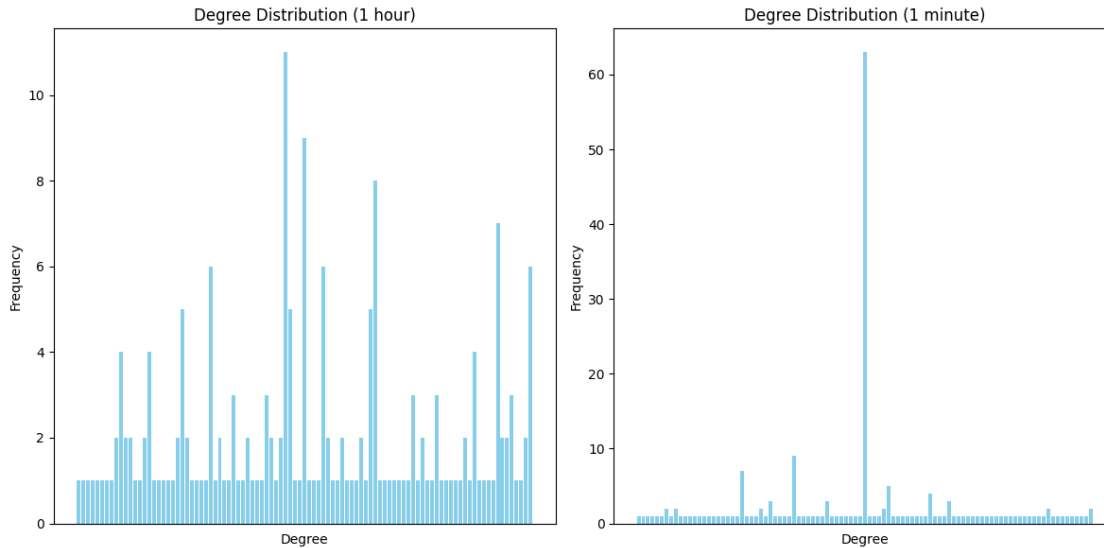
```
print("Top 5 stocks with highest degree in 1 minute MST:",  
      ↪list(sorted_dict_desc_1m.keys())[:5])
```

Top 5 stocks with highest degree in 1 hour MST: ['GOOG', 'HON', 'MMM', 'UNP', 'DIS']

Top 5 stocks with highest degree in 1 minute MST: ['HON', 'EMR', 'COP', 'JPM', 'MET']

2.3 3. Plot the degree distributions at 1 minute and 1 hour. Are the degree distributions qualitatively the same? Provide a comment to explain eventual similarities or dissimilarities.

```
[175]: # Plot degree distributions  
plt.figure(figsize=(12, 6))  
  
plt.subplot(1, 2, 1)  
plt.bar(degree_distribution_1h.keys(), degree_distribution_1h.values(),  
        ↪color='skyblue')  
plt.title('Degree Distribution (1 hour)')  
plt.xlabel('Degree')  
plt.ylabel('Frequency')  
plt.xticks([]) # Remove x-axis labels  
  
plt.subplot(1, 2, 2)  
plt.bar(degree_distribution_1m.keys(), degree_distribution_1m.values(),  
        ↪color='skyblue')  
plt.title('Degree Distribution (1 minute)')  
plt.xlabel('Degree')  
plt.ylabel('Frequency')  
plt.xticks([]) # Remove x-axis labels  
  
plt.tight_layout()  
plt.show()  
  
print("Sorted Dict which shows which node has which degree:")  
print(sorted_dict_desc_1h)  
print(sorted_dict_desc_1m)
```



Sorted Dict which shows which node has which degree:

```
{'GOOG': 11, 'HON': 9, 'MMM': 8, 'UNP': 7, 'DIS': 6, 'JNJ': 6, 'XOM': 6, 'COP': 5, 'GS': 5, 'MET': 5, 'APC': 4, 'BK': 4, 'T': 4, 'EMR': 3, 'GD': 3, 'NSC': 3, 'PG': 3, 'UTX': 3, 'APA': 2, 'AXP': 2, 'BA': 2, 'BIIB': 2, 'COF': 2, 'COST': 2, 'DVN': 2, 'FB': 2, 'GE': 2, 'GM': 2, 'JPM': 2, 'LMT': 2, 'MDLZ': 2, 'OXY': 2, 'SO': 2, 'UPS': 2, 'USB': 2, 'WMT': 2, 'AAPL': 1, 'ABV': 1, 'ABT': 1, 'ACN': 1, 'AIG': 1, 'ALL': 1, 'AMGN': 1, 'AMZN': 1, 'BAC': 1, 'BAX': 1, 'BMY': 1, 'C': 1, 'CAT': 1, 'CL': 1, 'CMCSA': 1, 'CSCO': 1, 'CVS': 1, 'CVX': 1, 'DD': 1, 'DOW': 1, 'EBAY': 1, 'EMC': 1, 'EXC': 1, 'F': 1, 'FCX': 1, 'FDX': 1, 'FOXA': 1, 'GILD': 1, 'HAL': 1, 'HD': 1, 'HPQ': 1, 'IBM': 1, 'INTC': 1, 'KO': 1, 'LLY': 1, 'LOW': 1, 'MA': 1, 'MCD': 1, 'MDT': 1, 'MO': 1, 'MON': 1, 'MRK': 1, 'MS': 1, 'MSFT': 1, 'NKE': 1, 'NOV': 1, 'ORCL': 1, 'PEP': 1, 'PFE': 1, 'PM': 1, 'QCOM': 1, 'RTN': 1, 'SBUX': 1, 'SLB': 1, 'SPG': 1, 'TGT': 1, 'TWX': 1, 'TXN': 1, 'UNH': 1, 'V': 1, 'VZ': 1}
{'HON': 63, 'EMR': 9, 'COP': 7, 'JPM': 5, 'MET': 4, 'DIS': 3, 'GD': 3, 'MRK': 3, 'AMGN': 2, 'APA': 2, 'CVX': 2, 'JNJ': 2, 'TXN': 2, 'XOM': 2, 'AAPL': 1, 'ABV': 1, 'ABT': 1, 'ACN': 1, 'AIG': 1, 'ALL': 1, 'AMZN': 1, 'APC': 1, 'AXP': 1, 'BA': 1, 'BAC': 1, 'BAX': 1, 'BIIB': 1, 'BK': 1, 'BMY': 1, 'C': 1, 'CAT': 1, 'CL': 1, 'CMCSA': 1, 'COF': 1, 'COST': 1, 'CSCO': 1, 'CVS': 1, 'DD': 1, 'DOW': 1, 'DVN': 1, 'EBAY': 1, 'EMC': 1, 'EXC': 1, 'F': 1, 'FB': 1, 'FCX': 1, 'FDX': 1, 'FOXA': 1, 'GE': 1, 'GILD': 1, 'GM': 1, 'GOOG': 1, 'GS': 1, 'HAL': 1, 'HD': 1, 'HPQ': 1, 'IBM': 1, 'INTC': 1, 'KO': 1, 'LLY': 1, 'LMT': 1, 'LOW': 1, 'MA': 1, 'MCD': 1, 'MDLZ': 1, 'MDT': 1, 'MMM': 1, 'MO': 1, 'MON': 1, 'MS': 1, 'MSFT': 1, 'NKE': 1, 'NOV': 1, 'NSC': 1, 'ORCL': 1, 'OXY': 1, 'PEP': 1, 'PFE': 1, 'PG': 1, 'PM': 1, 'QCOM': 1, 'RTN': 1, 'SBUX': 1, 'SLB': 1, 'SO': 1, 'SPG': 1, 'T': 1, 'TGT': 1, 'TWX': 1, 'UNH': 1, 'UNP': 1, 'UPS': 1, 'USB': 1, 'UTX': 1, 'V': 1, 'VZ': 1, 'WMT': 1}
```

Similarities: Both have a solid baseline of degree 1.

Dissimilarities: The minute degree plot has one clear maximum (HON) whereas the hour plot has various stocks with values over the baseline.

2.4 4. Compare the MST degree distributions of the 1 hour, 1 minute, Normal and One-Factor correlations: for each time scale, plot the three different distributions on the same plot. Is the degree distribution of MSTs well reproduced by the null Gaussian and One-Factor models?

```
[176]: len(arrays_nyse["cormat_gaussian_1h"])
```

```
[176]: 97
```

```
[177]: # Same as in 2.3 but in a function
def calculate_degree_distribution(data):
    G = nx.Graph()

    G.add_nodes_from(tickers)

    nx.set_node_attributes(G, tickers_dict, name='ticker')

    for i in range(len(tickers)):
        for j in range(i + 1, len(tickers)):
            weight = data[i][j]
            G.add_edge(tickers[i], tickers[j], weight=weight)

    mst = nx.minimum_spanning_tree(G)

    degree_distribution = dict(mst.degree())

    return degree_distribution
```

```
[178]: degree_distribution_gaussian_1h =
    ↪ calculate_degree_distribution(arrays_nyse["cormat_gaussian_1h"])
degree_distribution_gaussian_1m =
    ↪ calculate_degree_distribution(arrays_nyse["cormat_gaussian_1m"])
degree_distribution_onefactor_1h =
    ↪ calculate_degree_distribution(arrays_nyse["cormat_onefactor_1h"])
degree_distribution_onefactor_1m =
    ↪ calculate_degree_distribution(arrays_nyse["cormat_onefactor_1m"])
```

```
[181]: def plot_degree_distribution(degree_distribution, label):
    plt.hist(list(degree_distribution.values()), bins=20, alpha=0.5,
    ↪ label=label)

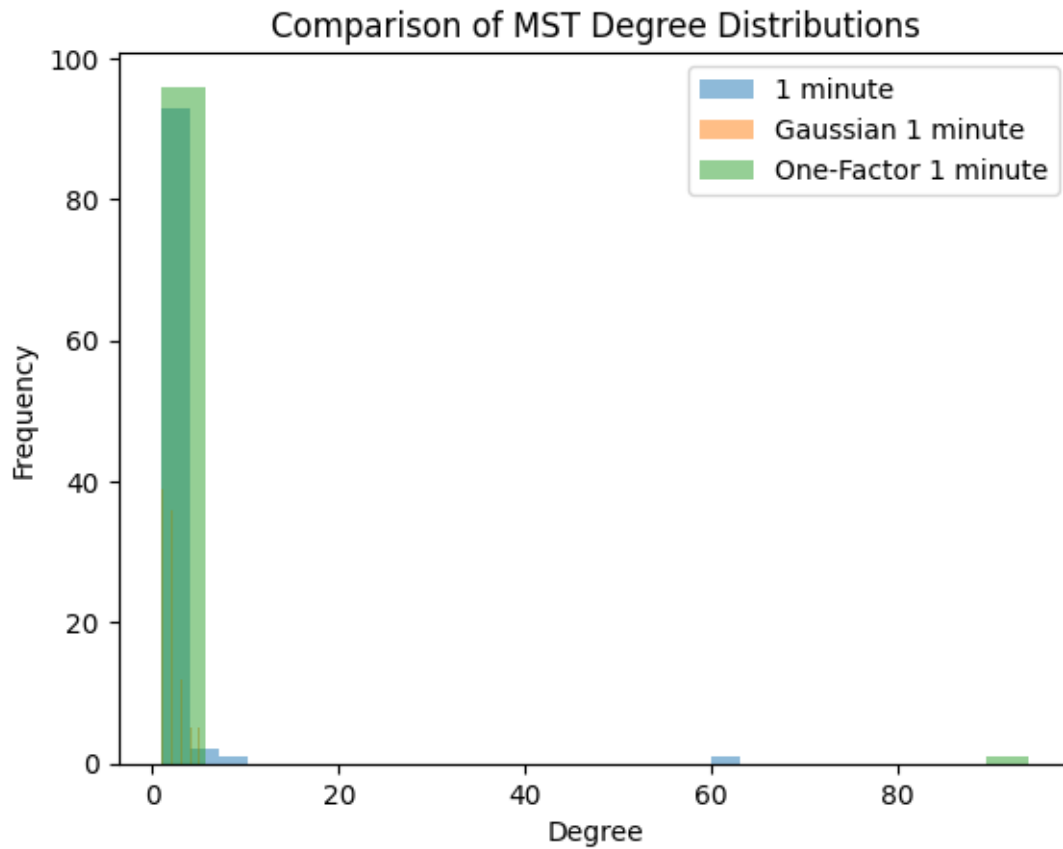
# Plot the degree distributions
plot_degree_distribution(degree_distribution_1m, '1 minute')
plot_degree_distribution(degree_distribution_gaussian_1m, 'Gaussian 1 minute')
```

```

plot_degree_distribution(degree_distribution_onefactor_1m, 'One-Factor 1_
↪minute')

# Add labels and legend
plt.legend()
plt.xlabel('Degree')
plt.ylabel('Frequency')
plt.title('Comparison of MST Degree Distributions')
plt.show()

```



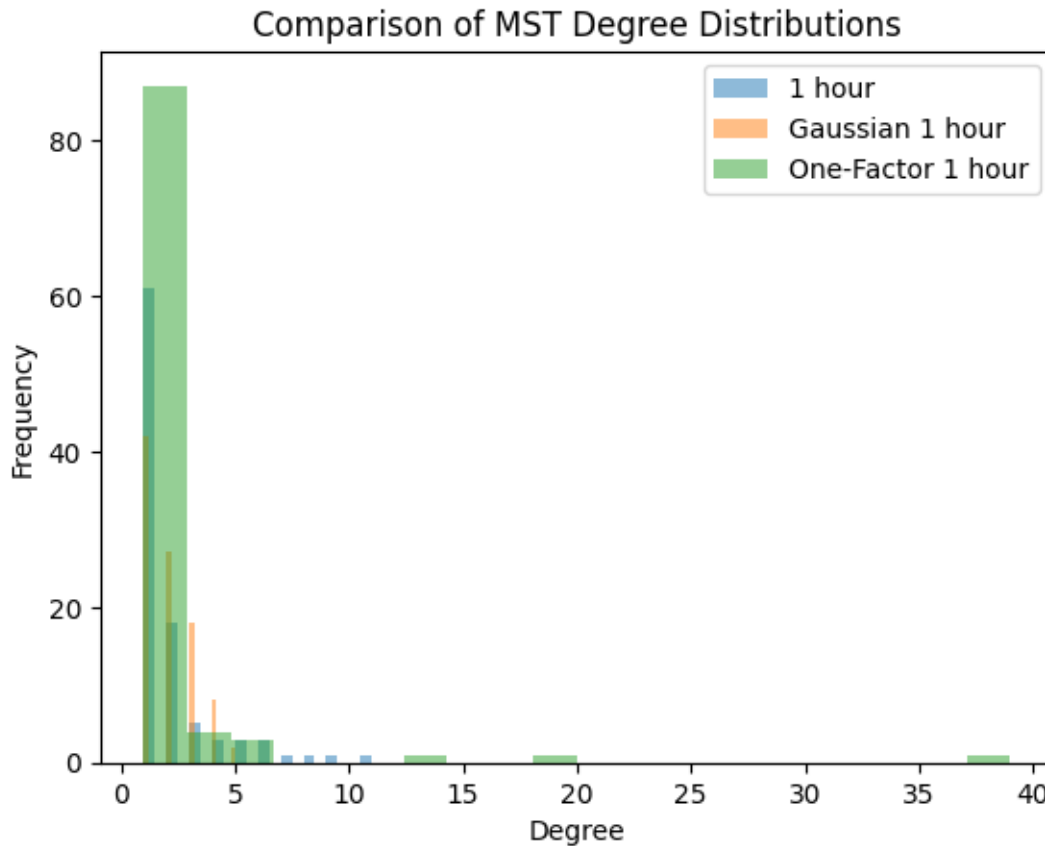
```

[182]: def plot_degree_distribution(degree_distribution, label):
        plt.hist(list(degree_distribution.values()), bins=20, alpha=0.5,
↪label=label)

# Plot the degree distributions
plot_degree_distribution(degree_distribution_1h, '1 hour')
plot_degree_distribution(degree_distribution_gaussian_1h, 'Gaussian 1 hour')
plot_degree_distribution(degree_distribution_onefactor_1h, 'One-Factor 1 hour')

```

```
# Add labels and legend
plt.legend()
plt.xlabel('Degree')
plt.ylabel('Frequency')
plt.title('Comparison of MST Degree Distributions')
plt.show()
```



```
[180]: print(dict(sorted(degree_distribution_1h.items(), key=lambda item: item[1],
    ↪reverse=True)))
print(dict(sorted(degree_distribution_1m.items(), key=lambda item: item[1],
    ↪reverse=True)))
print(dict(sorted(degree_distribution_onefactor_1h.items(), key=lambda item:
    ↪item[1], reverse=True)))
print(dict(sorted(degree_distribution_onefactor_1m.items(), key=lambda item:
    ↪item[1], reverse=True)))
print(dict(sorted(degree_distribution_gaussian_1h.items(), key=lambda item:
    ↪item[1], reverse=True)))
print(dict(sorted(degree_distribution_gaussian_1m.items(), key=lambda item:
    ↪item[1], reverse=True)))
```

```
print(dict(sorted(degree_distribution_onefactor_1h.items(), key=lambda item:
↪item[1], reverse=True)))
print(dict(sorted(degree_distribution_onefactor_1m.items(), key=lambda item:
↪item[1], reverse=True)))
```

```
{'GOOG': 11, 'HON': 9, 'MMM': 8, 'UNP': 7, 'DIS': 6, 'JNJ': 6, 'XOM': 6, 'COP':
5, 'GS': 5, 'MET': 5, 'APC': 4, 'BK': 4, 'T': 4, 'EMR': 3, 'GD': 3, 'NSC': 3,
'PG': 3, 'UTX': 3, 'APA': 2, 'AXP': 2, 'BA': 2, 'BIIB': 2, 'COF': 2, 'COST': 2,
'DVN': 2, 'FB': 2, 'GE': 2, 'GM': 2, 'JPM': 2, 'LMT': 2, 'MDLZ': 2, 'OXY': 2,
'SO': 2, 'UPS': 2, 'USB': 2, 'WMT': 2, 'AAPL': 1, 'ABV': 1, 'ABT': 1, 'ACN': 1,
'AIG': 1, 'ALL': 1, 'AMGN': 1, 'AMZN': 1, 'BAC': 1, 'BAX': 1, 'BMY': 1, 'C': 1,
'CAT': 1, 'CL': 1, 'CMCSA': 1, 'CSCO': 1, 'CVS': 1, 'CVX': 1, 'DD': 1, 'DOW': 1,
'EBAY': 1, 'EMC': 1, 'EXC': 1, 'F': 1, 'FCX': 1, 'FDX': 1, 'FOXA': 1, 'GILD': 1,
'HAL': 1, 'HD': 1, 'HPQ': 1, 'IBM': 1, 'INTC': 1, 'KO': 1, 'LLY': 1, 'LOW': 1,
'MA': 1, 'MCD': 1, 'MDT': 1, 'MO': 1, 'MON': 1, 'MRK': 1, 'MS': 1, 'MSFT': 1,
'NKE': 1, 'NOV': 1, 'ORCL': 1, 'PEP': 1, 'PFE': 1, 'PM': 1, 'QCOM': 1, 'RTN': 1,
'SBUX': 1, 'SLB': 1, 'SPG': 1, 'TGT': 1, 'TWX': 1, 'TXN': 1, 'UNH': 1, 'V': 1,
'VZ': 1}
```

```
{'HON': 63, 'EMR': 9, 'COP': 7, 'JPM': 5, 'MET': 4, 'DIS': 3, 'GD': 3, 'MRK': 3,
'AMGN': 2, 'APA': 2, 'CVX': 2, 'JNJ': 2, 'TXN': 2, 'XOM': 2, 'AAPL': 1, 'ABV':
1, 'ABT': 1, 'ACN': 1, 'AIG': 1, 'ALL': 1, 'AMZN': 1, 'APC': 1, 'AXP': 1, 'BA':
1, 'BAC': 1, 'BAX': 1, 'BIIB': 1, 'BK': 1, 'BMY': 1, 'C': 1, 'CAT': 1, 'CL': 1,
'CMCSA': 1, 'COF': 1, 'COST': 1, 'CSCO': 1, 'CVS': 1, 'DD': 1, 'DOW': 1, 'DVN':
1, 'EBAY': 1, 'EMC': 1, 'EXC': 1, 'F': 1, 'FB': 1, 'FCX': 1, 'FDX': 1, 'FOXA':
1, 'GE': 1, 'GILD': 1, 'GM': 1, 'GOOG': 1, 'GS': 1, 'HAL': 1, 'HD': 1, 'HPQ': 1,
'IBM': 1, 'INTC': 1, 'KO': 1, 'LLY': 1, 'LMT': 1, 'LOW': 1, 'MA': 1, 'MCD': 1,
'MDLZ': 1, 'MDT': 1, 'MMM': 1, 'MO': 1, 'MON': 1, 'MS': 1, 'MSFT': 1, 'NKE': 1,
'NOV': 1, 'NSC': 1, 'ORCL': 1, 'OXY': 1, 'PEP': 1, 'PFE': 1, 'PG': 1, 'PM': 1,
'QCOM': 1, 'RTN': 1, 'SBUX': 1, 'SLB': 1, 'SO': 1, 'SPG': 1, 'T': 1, 'TGT': 1,
'TWX': 1, 'UNH': 1, 'UNP': 1, 'UPS': 1, 'USB': 1, 'UTX': 1, 'V': 1, 'VZ': 1,
'WMT': 1}
```

```
{'PM': 39, 'WMT': 19, 'BAX': 13, 'EBAY': 6, 'IBM': 5, 'MA': 5, 'KO': 4, 'NKE':
4, 'F': 3, 'MCD': 3, 'AAPL': 2, 'CVX': 2, 'FDX': 2, 'T': 2, 'ABV': 1, 'ABT': 1,
'ACN': 1, 'AIG': 1, 'ALL': 1, 'AMGN': 1, 'AMZN': 1, 'APA': 1, 'APC': 1, 'AXP':
1, 'BA': 1, 'BAC': 1, 'BIIB': 1, 'BK': 1, 'BMY': 1, 'C': 1, 'CAT': 1, 'CL': 1,
'CMCSA': 1, 'COF': 1, 'COP': 1, 'COST': 1, 'CSCO': 1, 'CVS': 1, 'DD': 1, 'DIS':
1, 'DOW': 1, 'DVN': 1, 'EMC': 1, 'EMR': 1, 'EXC': 1, 'FB': 1, 'FCX': 1, 'FOXA':
1, 'GD': 1, 'GE': 1, 'GILD': 1, 'GM': 1, 'GOOG': 1, 'GS': 1, 'HAL': 1, 'HD': 1,
'HON': 1, 'HPQ': 1, 'INTC': 1, 'JNJ': 1, 'JPM': 1, 'LLY': 1, 'LMT': 1, 'LOW': 1,
'MDLZ': 1, 'MDT': 1, 'MET': 1, 'MMM': 1, 'MO': 1, 'MON': 1, 'MRK': 1, 'MS': 1,
'MSFT': 1, 'NOV': 1, 'NSC': 1, 'ORCL': 1, 'OXY': 1, 'PEP': 1, 'PFE': 1, 'PG': 1,
'QCOM': 1, 'RTN': 1, 'SBUX': 1, 'SLB': 1, 'SO': 1, 'SPG': 1, 'TGT': 1, 'TWX': 1,
'TXN': 1, 'UNH': 1, 'UNP': 1, 'UPS': 1, 'USB': 1, 'UTX': 1, 'V': 1, 'VZ': 1,
'XOM': 1}
```

```
{'F': 94, 'MA': 3, 'AAPL': 1, 'ABV': 1, 'ABT': 1, 'ACN': 1, 'AIG': 1, 'ALL': 1,
'AMGN': 1, 'AMZN': 1, 'APA': 1, 'APC': 1, 'AXP': 1, 'BA': 1, 'BAC': 1, 'BAX': 1,
'BIIB': 1, 'BK': 1, 'BMY': 1, 'C': 1, 'CAT': 1, 'CL': 1, 'CMCSA': 1, 'COF': 1,
'COP': 1, 'COST': 1, 'CSCO': 1, 'CVS': 1, 'CVX': 1, 'DD': 1, 'DIS': 1, 'DOW': 1,
```

'DVN': 1, 'EBAY': 1, 'EMC': 1, 'EMR': 1, 'EXC': 1, 'FB': 1, 'FCX': 1, 'FDX': 1,
 'FOXA': 1, 'GD': 1, 'GE': 1, 'GILD': 1, 'GM': 1, 'GOOG': 1, 'GS': 1, 'HAL': 1,
 'HD': 1, 'HON': 1, 'HPQ': 1, 'IBM': 1, 'INTC': 1, 'JNJ': 1, 'JPM': 1, 'KO': 1,
 'LLY': 1, 'LMT': 1, 'LOW': 1, 'MCD': 1, 'MDLZ': 1, 'MDT': 1, 'MET': 1, 'MMM': 1,
 'MO': 1, 'MON': 1, 'MRK': 1, 'MS': 1, 'MSFT': 1, 'NKE': 1, 'NOV': 1, 'NSC': 1,
 'ORCL': 1, 'OXY': 1, 'PEP': 1, 'PFE': 1, 'PG': 1, 'PM': 1, 'QCOM': 1, 'RTN': 1,
 'SBUX': 1, 'SLB': 1, 'SO': 1, 'SPG': 1, 'T': 1, 'TGT': 1, 'TWX': 1, 'TXN': 1,
 'UNH': 1, 'UNP': 1, 'UPS': 1, 'USB': 1, 'UTX': 1, 'V': 1, 'VZ': 1, 'WMT': 1,
 'XOM': 1}
 {'PG': 5, 'TWX': 5, 'FOXA': 4, 'GILD': 4, 'INTC': 4, 'MCD': 4, 'MON': 4, 'MS':
 4, 'MSFT': 4, 'UNH': 4, 'AIG': 3, 'AMGN': 3, 'AMZN': 3, 'BA': 3, 'BAC': 3,
 'BAX': 3, 'BK': 3, 'BMY': 3, 'CVX': 3, 'DIS': 3, 'DOW': 3, 'HAL': 3, 'JPM': 3,
 'LMT': 3, 'MA': 3, 'PEP': 3, 'QCOM': 3, 'SLB': 3, 'AAPL': 2, 'ACN': 2, 'ALL': 2,
 'BIIB': 2, 'C': 2, 'CAT': 2, 'CL': 2, 'COF': 2, 'COST': 2, 'CVS': 2, 'EBAY': 2,
 'EMC': 2, 'FDX': 2, 'JNJ': 2, 'LLY': 2, 'MDLZ': 2, 'MMM': 2, 'NKE': 2, 'NOV': 2,
 'NSC': 2, 'SO': 2, 'T': 2, 'TXN': 2, 'UNP': 2, 'USB': 2, 'UTX': 2, 'VZ': 2,
 'ABBV': 1, 'ABT': 1, 'APA': 1, 'APC': 1, 'AXP': 1, 'CMCSA': 1, 'COP': 1, 'CSCO':
 1, 'DD': 1, 'DVN': 1, 'EMR': 1, 'EXC': 1, 'F': 1, 'FB': 1, 'FCX': 1, 'GD': 1,
 'GE': 1, 'GM': 1, 'GOOG': 1, 'GS': 1, 'HD': 1, 'HON': 1, 'HPQ': 1, 'IBM': 1,
 'KO': 1, 'LOW': 1, 'MDT': 1, 'MET': 1, 'MO': 1, 'MRK': 1, 'ORCL': 1, 'OXY': 1,
 'PFE': 1, 'PM': 1, 'RTN': 1, 'SBUX': 1, 'SPG': 1, 'TGT': 1, 'UPS': 1, 'V': 1,
 'WMT': 1, 'XOM': 1}
 {'DD': 5, 'JNJ': 5, 'TGT': 5, 'TXN': 5, 'UTX': 5, 'AMZN': 4, 'EMR': 4, 'GE': 4,
 'IBM': 4, 'USB': 4, 'APA': 3, 'APC': 3, 'BK': 3, 'COF': 3, 'DVN': 3, 'F': 3,
 'GM': 3, 'INTC': 3, 'NKE': 3, 'TWX': 3, 'UNH': 3, 'V': 3, 'AAPL': 2, 'ABT': 2,
 'AIG': 2, 'ALL': 2, 'AXP': 2, 'BAC': 2, 'BAX': 2, 'C': 2, 'CL': 2, 'CMCSA': 2,
 'COP': 2, 'CVS': 2, 'EMC': 2, 'EXC': 2, 'FB': 2, 'FCX': 2, 'FOXA': 2, 'GOOG': 2,
 'GS': 2, 'HD': 2, 'HON': 2, 'JPM': 2, 'LLY': 2, 'LMT': 2, 'MCD': 2, 'MDLZ': 2,
 'MET': 2, 'MRK': 2, 'PEP': 2, 'PG': 2, 'PM': 2, 'SLB': 2, 'SO': 2, 'T': 2,
 'UPS': 2, 'VZ': 2, 'ABBV': 1, 'ACN': 1, 'AMGN': 1, 'BA': 1, 'BIIB': 1, 'BMY': 1,
 'CAT': 1, 'COST': 1, 'CSCO': 1, 'CVX': 1, 'DIS': 1, 'DOW': 1, 'EBAY': 1, 'FDX':
 1, 'GD': 1, 'GILD': 1, 'HAL': 1, 'HPQ': 1, 'KO': 1, 'LOW': 1, 'MA': 1, 'MDT': 1,
 'MMM': 1, 'MO': 1, 'MON': 1, 'MS': 1, 'MSFT': 1, 'NOV': 1, 'NSC': 1, 'ORCL': 1,
 'OXY': 1, 'PFE': 1, 'QCOM': 1, 'RTN': 1, 'SBUX': 1, 'SPG': 1, 'UNP': 1, 'WMT':
 1, 'XOM': 1}
 {'PM': 39, 'WMT': 19, 'BAX': 13, 'EBAY': 6, 'IBM': 5, 'MA': 5, 'KO': 4, 'NKE':
 4, 'F': 3, 'MCD': 3, 'AAPL': 2, 'CVX': 2, 'FDX': 2, 'T': 2, 'ABBV': 1, 'ABT': 1,
 'ACN': 1, 'AIG': 1, 'ALL': 1, 'AMGN': 1, 'AMZN': 1, 'APA': 1, 'APC': 1, 'AXP':
 1, 'BA': 1, 'BAC': 1, 'BIIB': 1, 'BK': 1, 'BMY': 1, 'C': 1, 'CAT': 1, 'CL': 1,
 'CMCSA': 1, 'COF': 1, 'COP': 1, 'COST': 1, 'CSCO': 1, 'CVS': 1, 'DD': 1, 'DIS':
 1, 'DOW': 1, 'DVN': 1, 'EMC': 1, 'EMR': 1, 'EXC': 1, 'FB': 1, 'FCX': 1, 'FOXA':
 1, 'GD': 1, 'GE': 1, 'GILD': 1, 'GM': 1, 'GOOG': 1, 'GS': 1, 'HAL': 1, 'HD': 1,
 'HON': 1, 'HPQ': 1, 'INTC': 1, 'JNJ': 1, 'JPM': 1, 'LLY': 1, 'LMT': 1, 'LOW': 1,
 'MDLZ': 1, 'MDT': 1, 'MET': 1, 'MMM': 1, 'MO': 1, 'MON': 1, 'MRK': 1, 'MS': 1,
 'MSFT': 1, 'NOV': 1, 'NSC': 1, 'ORCL': 1, 'OXY': 1, 'PEP': 1, 'PFE': 1, 'PG': 1,
 'QCOM': 1, 'RTN': 1, 'SBUX': 1, 'SLB': 1, 'SO': 1, 'SPG': 1, 'TGT': 1, 'TWX': 1,
 'TXN': 1, 'UNH': 1, 'UNP': 1, 'UPS': 1, 'USB': 1, 'UTX': 1, 'V': 1, 'VZ': 1,
 'XOM': 1}

```
{'F': 94, 'MA': 3, 'AAPL': 1, 'ABBV': 1, 'ABT': 1, 'ACN': 1, 'AIG': 1, 'ALL': 1,
'AMGN': 1, 'AMZN': 1, 'APA': 1, 'APC': 1, 'AXP': 1, 'BA': 1, 'BAC': 1, 'BAX': 1,
'BIIB': 1, 'BK': 1, 'BMY': 1, 'C': 1, 'CAT': 1, 'CL': 1, 'CMCSA': 1, 'COF': 1,
'COP': 1, 'COST': 1, 'CSCO': 1, 'CVS': 1, 'CVX': 1, 'DD': 1, 'DIS': 1, 'DOW': 1,
'DVN': 1, 'EBAY': 1, 'EMC': 1, 'EMR': 1, 'EXC': 1, 'FB': 1, 'FCX': 1, 'FDX': 1,
'FOXA': 1, 'GD': 1, 'GE': 1, 'GILD': 1, 'GM': 1, 'GOOG': 1, 'GS': 1, 'HAL': 1,
'HD': 1, 'HON': 1, 'HPQ': 1, 'IBM': 1, 'INTC': 1, 'JNJ': 1, 'JPM': 1, 'KO': 1,
'LLY': 1, 'LMT': 1, 'LOW': 1, 'MCD': 1, 'MDLZ': 1, 'MDT': 1, 'MET': 1, 'MMM': 1,
'MO': 1, 'MON': 1, 'MRK': 1, 'MS': 1, 'MSFT': 1, 'NKE': 1, 'NOV': 1, 'NSC': 1,
'ORCL': 1, 'OXY': 1, 'PEP': 1, 'PFE': 1, 'PG': 1, 'PM': 1, 'QCOM': 1, 'RTN': 1,
'SBUX': 1, 'SLB': 1, 'SO': 1, 'SPG': 1, 'T': 1, 'TGT': 1, 'TWX': 1, 'TXN': 1,
'UNH': 1, 'UNP': 1, 'UPS': 1, 'USB': 1, 'UTX': 1, 'V': 1, 'VZ': 1, 'WMT': 1,
'XOM': 1}
```

Yes they are relatively similar. But we can see that on both plots (hour and minute) the OneFactor has a more extended tail than the empiric one. Also the Gaussian is shorter than the other ones. However they have the same structure!

Creating Results: jupyter nbconvert Assignment_03.ipynb --to pdf --output
Assignment_03_Results-T2.pdf